

Kubernetes – GitOps avec ArgoCD et Helm

Jour 4 — Helm

Packaging et distribution d'applications Kubernetes

Kubernetes — GitOps avec ArgoCD et Helm — m2i

Module 4/5 (Module 2 / Jour 1)

Objectifs du module

À la fin de cette journée, vous serez capable de :

- Expliquer pourquoi Helm est nécessaire au-delà de quelques manifestes
- Installer, mettre à jour, rollback un chart public
- Lire et écrire un chart de zéro (Chart.yaml, values, templates, helpers)
- Templater du multi-environnements (dev / staging / prod)
- Packager et publier un chart sur un dépôt OCI

Durée : 7 heures (1 journée)

Plan

1. Pourquoi Helm ?
2. Utiliser des charts existants
3. Anatomie d'un chart
4. Templating Helm
5. Gestion multi-environnements
6. Publication et bonnes pratiques
7. Démonstrations et TP

1. Pourquoi Helm ?

Le problème : `kubectl apply` ne tient pas la prod

Ce qui marche en TP

- 5 manifestes
- 1 environnement
- 1 mainteneur

Ce qui casse en prod

- 50 manifestes par app
- 3 à 10 environnements
- Plusieurs équipes
- Versionnement à gérer
- Rollback en 1 commande exigé

Positionnement de Helm

“ "Helm est le gestionnaire de paquets de Kubernetes." ”

Trois besoins comblés :

- **Paramétrer** : un même template, N environnements
- **Versionner** : chart en SemVer, release historisée
- **Distribuer** : un fichier `.tgz` consommable par tous

Inspiré d'apt/yum/npm. Mais surtout : Helm est une convention partagée par toute la CNCF.

Architecture Helm 3

```
+-----+ +-----+
| helm CLI (local) | -----> | cluster Kubernetes |
| - Chart.yaml    |           | - secrets release   |
| - values.yaml   |           | - apiserver         |
| - templates/    |           | - objets natifs     |
+-----+ +-----+
```

Pas de Tiller (mort en Helm 2). Tout est côté client + Secrets dans le cluster pour tracer les releases.

Vocabulaire à fixer

Terme	Définition
chart	Package : <code>Chart.yaml</code> , <code>values.yaml</code> , <code>templates/</code>
release	Installation d'un chart dans un namespace, avec un nom
repository	Dépôt de charts (HTTP ou OCI)
values	Paramètres passés à un chart

Une release = `chart × valeurs × cluster × namespace × moment`.

2. Utiliser des charts existants

Trouver un chart

```
helm search hub postgresql          # Artifact Hub (recherche large)
helm repo add bitnami https://charts.bitnami.com/bitnami
helm search repo postgres           # recherche dans repos ajoutés
helm show values bitnami/postgresql # voir toutes les values
helm show readme bitnami/postgresql # doc
```

Artifact Hub est le moteur unifié (charts + opérateurs Helm).

Installer, mettre à jour, désinstaller

```
helm install pg bitnami/postgresql -n data --create-namespace \
  --set auth.postgresPassword=ChangeMe123

helm list -n data
helm get values pg -n data
helm get manifest pg -n data      # YAML rendu

helm upgrade pg bitnami/postgresql -n data \
  --reuse-values --set primary.persistence.size=20Gi

helm uninstall pg -n data
```

Historique et rollback

```
helm history pg -n data
# REVISION  STATUS    CHART           ACTION
# 1         deployed  postgresql-15   install
# 2         deployed  postgresql-15   upgrade
# 3         failed    postgresql-15   upgrade

helm rollback pg 2 -n data
helm history pg -n data
# 4         deployed  postgresql-15   rollback to 2
```

Le rollback crée une **nouvelle** révision, il n'efface jamais l'historique.

Atomic et timeout

```
helm install pg bitnami/postgresql -n data \  
  --atomic --timeout 5m
```

- `--atomic` : annule automatiquement en cas d'erreur
- `--timeout` : limite l'attente d'état Ready

À mettre dans tous les pipelines CI/CD : évite les half-deploys.

Dépôts OCI vs HTTP

HTTP (legacy)

- `index.yaml` + `.tgz`
- Simple à héberger
- Pas de pull authentifié natif

OCI (recommandé)

- Stocké comme une image Docker
- Authentification = registry classique
- Outillage commun (Harbor, ECR, GHCR)

```
helm push my-chart-0.1.0.tgz oci://registry.m2i.local/charts
helm install demo oci://registry.m2i.local/charts/my-chart --version 0.1.0
```

3. Anatomie d'un chart

Structure standard

```
demo-api-chart/  
  Chart.yaml          # métadonnées  
  values.yaml         # valeurs par défaut  
  templates/  
    _helpers.tpl      # helpers réutilisables  
    deployment.yaml  
    service.yaml  
    ingress.yaml  
    configmap.yaml  
    NOTES.txt         # message post-install  
  charts/             # sous-charts (dépendances)  
  .helmignore
```

C'est exactement la structure de `apps/demo-api-chart/` de la formation.

Chart.yaml

```
apiVersion: v2
name: demo-api
description: Chart Helm pour l'application demo-api Spring Boot
type: application
version: 0.1.0           # version du chart (SemVer)
appVersion: "1.0.0"      # version de l'application packagée
keywords: [spring-boot, java, m2i]
maintainers:
  - name: Utopios
    email: contact@utopios.net
dependencies:
  - name: postgresql
    version: "15.5.0"
    repository: "https://charts.bitnami.com/bitnami"
    condition: postgresql.enabled
```

Version et appVersion bougent indépendamment.

Hooks Helm

```
metadata:
  annotations:
    "helm.sh/hook": pre-install,pre-upgrade
    "helm.sh/hook-weight": "-5"
    "helm.sh/hook-delete-policy": before-hook-creation,hook-succeeded
```

Hook	Cas d'usage
pre-install / pre-upgrade	Migration DB, vérification environnement
post-install / post-upgrade	Seed de données, configuration tierce
test	Smoke test (helm test)
pre-delete / post-delete	Cleanup ordonné

NOTES.txt

```
demo-api {{ .Chart.AppVersion }} a été installé dans {{ .Release.Namespace }}.
```

Accès :

```
{{- if .Values.ingress.enabled }}  
  http://{{ (index .Values.ingress.hosts 0).host }}  
{{- else }}  
  kubectl port-forward svc/{{ include "demo-api.fullname" . }} 8080:80  
{{- end }}
```

Affiché à la fin de `helm install`. Doit donner les 3 prochaines actions de l'utilisateur.

4. Templating Helm

Le moteur Go templates

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "demo-api.fullname" . }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
        - image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          env:
            {{- range $k, $v := .Values.env }}
            - name: {{ $k }}
              value: {{ $v | quote }}
            {{- end }}
```

`{{ }}` délimite une action template. `{{- }}` mange le whitespace avant/après.

Variables intégrées

Variable	Contenu
<code>.Values</code>	Le contenu de <code>values.yaml</code> (fusionné avec <code>--set</code> / <code>-f</code>)
<code>.Release</code>	<code>.Name</code> , <code>.Namespace</code> , <code>.Revision</code> , <code>.Service</code>
<code>.Chart</code>	<code>.Name</code> , <code>.Version</code> , <code>.AppVersion</code>
<code>.Files</code>	Accès aux fichiers du chart (utile pour <code>_helpers</code>)
<code>.Capabilities</code>	Version K8s, APIs disponibles

Fonctions et pipelines

```
# default : valeur par défaut si la value est nil
image: "{{ .Values.image | default \"nginx:latest\" }}"

# required : erreur si la value est absente
password: "{{ required \"db password is required\" .Values.db.password }}"

# quote, indent, nindent : formatage
data: |
  {{ .Values.config | nindent 4 }}

# toYaml : injecter un sous-objet
resources:
  {{- toYaml .Values.resources | nindent 4 }}
```

À retenir : `nindent N` est plus sûr qu'`indent N` (newline + indent).

Contrôle de flux

```
{{- if .Values.ingress.enabled }}
apiVersion: networking.k8s.io/v1
kind: Ingress
spec:
  rules:
    {{- range .Values.ingress.hosts }}
    - host: {{ .host | quote }}
      http:
        paths:
          {{- range .paths }}
          - path: {{ .path }}
            pathType: {{ .pathType }}
          {{- end }}
    {{- end }}
  {{- end }}
```

if/else, range, with couvrent 90% des besoins.

Helpers réutilisables

templates/_helpers.tpl :

```
{{- define "demo-api.fullname" -}}
{{- $name := default .Chart.Name .Values.nameOverride -}}
{{- printf "%s-%s" .Release.Name $name | trunc 63 | trimSuffix "-" -}}
{{- end -}}

{{- define "demo-api.labels" -}}
helm.sh/chart: {{ printf "%s-%s" .Chart.Name .Chart.Version }}
app.kubernetes.io/name: {{ include "demo-api.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
{{- end -}}
```

Le préfixe `_` empêche le rendu direct. Utiliser via `{{ include "demo-api.fullname" . }}`.

Valeurs sensibles : ce qu'il faut savoir

“ Ne JAMAIS committer une valeur sensible en clair dans un chart Helm. ”

Solutions standards :

- **SealedSecrets** (Bitnami) : Secret chiffré dans Git, déchiffré par un controller in-cluster
- **External Secrets Operator** : pull depuis Vault, AWS Secrets Manager, etc.
- **--set-string** côté CI/CD avec un secret store

Hors-scope de l'installation mais à mentionner systématiquement.

5. Gestion multi-environnements

Le pattern : un fichier par environnement

```
demo-api-chart/  
  values.yaml           # defaults sans surprise  
  values-dev.yaml       # surcharges dev  
  values-staging.yaml  
  values-prod.yaml
```

```
helm install demo-api ./demo-api-chart \  
-f ./demo-api-chart/values.yaml \  
-f ./demo-api-chart/values-prod.yaml \  
-n prod --create-namespace
```

Plus à droite = plus fort. La CLI (`--set`) bat tout.

Exemple values-prod.yaml

```
replicaCount: 3
env:
  LOG_LEVEL: "WARN"
ingress:
  enabled: true
  tls:
    - hosts: [demo-api.m2i.local]
      secretName: demo-api-tls
resources:
  requests: {cpu: 500m, memory: 512Mi}
  limits:   {cpu: 2000m, memory: 1Gi}
autoscaling:
  enabled: true
  minReplicas: 3
  maxReplicas: 10
```

Ne contient QUE les surcharges. Le reste vient de `values.yaml`.

Umbrella charts et sous-charts

```
my-app/  
  Chart.yaml  
  values.yaml  
  templates/  
  charts/           # sous-charts (dépendances)  
    demo-api/       # via dependencies dans Chart.yaml  
    postgresql/
```

```
helm dependency update ./my-app  
helm install my-app ./my-app
```

Une seule release qui déploie plusieurs charts. Inversement, on évite quand les composants ont des cycles de vie indépendants.

6. Publication et bonnes pratiques

Lint et tests

```
helm lint ./demo-api-chart           # syntaxe + warnings
helm template demo-api ./demo-api-chart --debug    # rendu sans cluster
helm install demo-api ./demo-api-chart --dry-run   # rendu avec validation API
helm test demo-api -n dev              # exécute les pods de test
```

Pipeline CI typique : `lint` puis `template` puis `install --dry-run`.

Packaging

```
helm package ./demo-api-chart -d ./dist  
# crée ./dist/demo-api-0.1.0.tgz  
  
helm push ./dist/demo-api-0.1.0.tgz \  
oci://registry.m2i.local/charts
```

Une release par commit. Bumper la version du chart à chaque changement de comportement.

SemVer pour les charts

Type de changement	Bump
Bug fix sans changement de signature	0.1.0 → 0.1.1
Nouvelle value avec default rétro-compatible	0.1.1 → 0.2.0
Suppression de value, renommage breaking	0.2.0 → 1.0.0

`appVersion` suit la version de l'application packagée, indépendamment du chart.

Labels standards Kubernetes

```
labels:  
  app.kubernetes.io/name: demo-api  
  app.kubernetes.io/instance: demo-api-prod  
  app.kubernetes.io/version: "1.2.3"  
  app.kubernetes.io/component: api  
  app.kubernetes.io/part-of: m2i-platform  
  app.kubernetes.io/managed-by: Helm
```

Standard officiel. À mettre dans tous les charts via un helper.

Démo live

Démo 1 — Installer Bitnami PostgreSQL avec des values personnalisées

Voir `demos/demo-01.md`

Démo live

Démo 2 — Créer un chart from scratch et l'installer

Voir `demos/demo-02.md`

Démo live

Démo 3 — Templating avancé et multi-environnements

Voir `demos/demo-03.md`

Démo live

Démo 4 — Packaging et publication OCI

Voir `demos/demo-04.md`

TP à suivre

TP 1 — Installer chart Bitnami PostgreSQL · 30 min · `tp/tp-01.md`

TP 2 — Créer un chart Helm pour `demo-api` · 75 min · `tp/tp-02.md`

TP 3 — Ajouter un sidecar Python conditionnel · 45 min · `tp/tp-03.md`

TP 4 — Multi-environnements dev/staging/prod · 45 min · `tp/tp-04.md`

TP 5 — Packaging et publication OCI · 30 min · `tp/tp-05.md`

Récap du module

- Helm = paramétrage + versionnement + distribution
- Un chart = `Chart.yaml` + `values.yaml` + `templates/` + helpers
- Templating Go : `{{ }}`, fonctions, `if/range/with`, helpers
- Un fichier de values par environnement, fusionnés à l'install
- Lint + template + dry-run dans la CI, puis package et push OCI

Cheatsheet : `cheatsheet/module-04.md`

Questions

Demain : GitOps avec ArgoCD

